

Relazione di Progetto: Bomberman TUI

Sviluppatore del Progetto

September 3, 2026

Abstract

La presente relazione illustra le fasi di sviluppo del progetto "Bomberman TUI" in C++ con libreria ncurses. Il documento mette in luce il processo evolutivo del codice, confrontando le idee e le implementazioni iniziali con le soluzioni finali adottate per risolvere bug tecnici, migliorare l'efficienza e rispettare i rigidi vincoli accademici imposti.

Indice

1 Introduzione

Lo sviluppo del progetto ha richiesto di replicare le dinamiche classiche di Bomberman all'interno di un'interfaccia a terminale (TUI) usando `ncurses`. Tra le restrizioni più stringenti vi era il divieto di usare costrutti moderni della Standard Template Library (es. `std::vector`), il divieto di usare la parola chiave `private` (sostituita da `protected`) e il divieto di utilizzare distruttori espliciti, rendendo la gestione della memoria una sfida puramente manuale.

2 Generazione della Mappa e Rendering Visivo

2.1 Cosa ho implementato

Il problema principale era disegnare la mappa di gioco (40x20) rendendola esteticamente identica al video di riferimento fornito dai docenti. Inizialmente avevo pensato di usare caratteri ASCII classici come `#` per i muri e `%` per le casse, e applicare colori basilari. Avevo anche tentato di usare dei caratteri UTF-8 speciali e costrutti `ncurses` come `ACS_BLOCK` per simulare blocchi pieni.

2.2 Il mio codice (Idee Iniziali)

Inizialmente avevo definito la matrice in modo invertito e usato una logica complessa per disegnare i blocchi a schermo:

```
1 // Errore logico nella definizione iniziale
2 char griglia[40][20];
3
4 // Tentativo iniziale di renderizzare i blocchi
5 if (blocco == '#') {
6     attron(COLOR_PAIR(5) | A_BOLD);
7     addch(ACS_BLOCK);
8     attroff(COLOR_PAIR(5) | A_BOLD);
9 }
```

Avevo anche provato a generare i blocchi usando griglie geometriche rigorose (es. muri solo in posizioni pari) o formule matematiche fisse per la difficoltà.

2.3 Eventuali problemi

Questa implementazione presentava tre gravi problemi:

1. **Corruzione Memoria:** Dichiarare `griglia[40][20]` e poi scorrere la larghezza fino a 40 (che per il C++ era l'indice delle colonne, impostato a 20) causava un accavallamento di memoria, distorcendo l'intera mappa a schermo.
2. **Instabilità Visiva:** L'uso combinato di `ACS_BLOCK`, macro complesse e UTF-8 creava artefatti grafici su terminali diversi, rendendo la cornice e i muri "rotti".
3. **Level Design Errato:** Applicando una griglia rigida, il gioco non rispecchiava il video demo del professore, in cui i muri solidi sono distribuiti in modo pseudo-casuale (a cluster). Generarli in modo puramente casuale senza limiti rischiava però di creare mappe al 100% bloccate.

2.4 Soluzione migliorata

Insieme abbiamo corretto l'allineamento della matrice tornando a `griglia[20][40]`. Per il rendering visivo, abbiamo sostituito l'instabile `ACS_BLOCK` con una tecnica estremamente robusta: stampare uno spazio vuoto (' ') assegnandogli però un `COLOR_PAIR` in cui lo sfondo è bianco (o grigio).

Per la generazione del livello, abbiamo implementato un algoritmo puramente pseudo-casuale fedele al video del professore, integrando però uno `switch-case` che modula le percentuali (densità) in base al livello per evitare soffocamenti:

```
1 // Rendering Sicuro dei Muri Solidi
2 if (elemento == '#') {
3     attron(COLOR_PAIR(5)); // Sfondo bianco, testo nero
4     addch(' ');
5     attroff(COLOR_PAIR(5));
6 }
7
8 // ... nello switch del level design:
9 case 5: // Livello 5: Difficile ma non asfissiante
10     densitaSolidi = 15; // Massimo 15% di muri indistruttibili
11     densitaCasse = 30; // Massimo 30% di casse
12     numeroTotaleNemici = 10;
13     break;
```

2.5 Motivazioni delle scelte

Si è scelto lo "spazio colorato" rispetto ai macro `ACS_*` per garantire compatibilità al 100% su qualsiasi terminale Linux usato per la correzione. Lo `switch-case` asimmetrico per il rateo di spawn assicura che il gioco aderisca al video ufficiale dei tutor (niente scacchiera fissa) mantenendo però un gameplay bilanciato e sempre risolvibile.

2.6 Come lo spiegherei all'esame

"Professore, inizialmente avevo provato a disegnare i blocchi usando macro speciali di ncurses o caratteri ASCII avanzati, ma mi sono accorto che su alcuni terminali si rompeva la formattazione. Ho quindi optato per una soluzione più elegante e sicura: stampare dei semplici caratteri 'spazio', ma colorando il background tramite i `COLOR_PAIR`.

Per la generazione della mappa, dopo aver analizzato a fondo il vostro video dimostrativo, ho notato che i muri non formano una scacchiera perfetta. Ho quindi implementato un algoritmo pseudo-casuale. Per evitare che la mappa generata fosse 'ingiocabile', ho legato la probabilità di spawn di muri e casse a uno `switch` basato sull'id del livello."

3 Gestione Livelli: Lista Bidirezionale

3.1 Cosa ho implementato

Il progetto richiede la presenza di 5 livelli, con la possibilità di spostarsi al livello successivo o precedente tramite input utente ('n' e 'p'). Dato il divieto di usare `std::vector`, dovevo trovare un modo per immagazzinare e navigare tra le mappe.

3.2 Il mio codice (Idee Iniziali)

Sapevo di dover usare una lista concatenata, ma gestire correttamente i puntatori avanti e indietro in C++ senza leak di memoria e senza usare i costruttori/distruttori standard in modo moderno mi creava insicurezza.

3.3 Eventuali problemi

Il rischio maggiore era lasciare memoria appesa (memory leak) alla chiusura del programma, dato il vincolo del professore che vieta esplicitamente l'uso di distruttori automatici nelle classi.

3.4 Soluzione migliorata

Abbiamo progettato una classe `GestoreLivelli` che incapsula la logica della lista. I nodi della lista sono gli stessi oggetti `Mappa`, a cui abbiamo aggiunto puntatori pubblici `next` e `prev`.

La deallocazione è stata implementata in modo rigorosamente manuale e iterativo alla fine del `main()`:

```
1 Mappa* temp = gestore.getTesta();
2 while (temp != nullptr) {
3     Mappa* daCancellare = temp;
4     temp = temp->next;
5     delete daCancellare; // Deallocazione manuale esplicita
6 }
```

3.5 Motivazioni delle scelte

Questa architettura soddisfa due requisiti critici: le prestazioni (navigare ai livelli adiacenti costa $O(1)$) e il pieno rispetto dei vincoli accademici. Aver spostato la deallocazione nel `main` rende palese al correttore che la memoria è stata gestita consapevolmente e non delegata a distruttori nascosti.

3.6 Come lo spiegherei all'esame

*"Visto il divieto di usare la Standard Template Library, ho strutturato i livelli come una vera e propria lista doppiamente concatenata. Ogni oggetto **Mappa** è un nodo che punta al livello successivo e precedente. La vera particolarità è la gestione della memoria: rispettando il divieto di usare i distruttori di classe, ho scritto un ciclo dedicato nel **main** che intercetta il momento esatto in cui il gioco si chiude, scorre l'intera lista partendo dalla testa e usa il comando **delete** per liberare manualmente la memoria allocata con **new** per ogni singola mappa."*

4 Sistema di Collisioni

4.1 Cosa ho implementato

Per permettere il movimento del giocatore e bloccarlo contro i muri, dovevo implementare un sistema di collisioni.

4.2 Il mio codice (Idee Iniziali)

La mia prima idea era quella di creare una **seconda matrice numerica** (di interi) nascosta dietro quella visiva, in cui ogni posizione aveva un valore numerico:

- 0 → Nessun muro (casella libera)
- 1 → Muro distruttibile
- -1 → Muro indistruttibile

L'idea era di controllare questa matrice numerica prima di ogni spostamento.

4.3 Eventuali problemi

Creare una matrice di interi separata per le collisioni è un approccio inefficiente. Violerebbe il principio DRY (Don't Repeat Yourself). Inoltre, creerebbe gravissimi problemi di sincronizzazione: se una bomba esplode e distrugge una cassa, bisognerebbe ricordarsi di eliminare la cassa dalla matrice visiva e aggiornare contemporaneamente l'1 a 0 nella matrice intera. Una mancata sincronizzazione permetterebbe al giocatore di camminare attraverso muri apparentemente intatti o bloccarsi nel vuoto.

4.4 Soluzione migliorata

Invece di creare nuove strutture dati, abbiamo riutilizzato la matrice visiva `char griglia[20][40]` esistente come nostra mappa di collisione universale. Abbiamo implementato in **Mappa dei getter/setter sicuri**:

```
1 char Mappa::getCella(int y, int x) {
2     // Controllo dei confini per evitare Segmentation Fault
3     if (y >= 0 && y < altezza && x >= 0 && x < larghezza) {
4         return griglia[y][x];
5     }
6     return '#'; // Oltre il confine muro
7 }
```

Quando il Giocatore calcola il suo spostamento in base all'input, gli basta interrogare la mappa: se `getCella` restituisce ' ', si sposta; altrimenti se restituisce '#' o '%' si blocca.

4.5 Motivazioni delle scelte

Leggere direttamente lo stato logico della mappa visiva garantisce la "Single Source of Truth" (Singola Fonte di Verità). Nessun rischio di desincronizzazione, meno memoria occupata, e complessità del codice dimezzata. I check (`y < altezza` ecc.) evitano inoltre i classici errori di *segmentation fault* quando i personaggi toccano il bordo dello schermo.

4.6 Come lo spiegherei all'esame

"Per le collisioni, inizialmente avevo pensato di creare una seconda matrice di interi riempita con 0, 1 e -1 per calcolare ostacoli e muri. Riflettendoci, ho capito che era ridondante e pericoloso: avrei dovuto mantenere sincronizzate due matrici ogni volta che una bomba esplodeva. La mia soluzione definitiva è stata quella di usare la matrice `char griglia` stessa come unica fonte di verità. Ho creato un metodo `getCella()` che viene richiamato dall'entità in movimento. Il giocatore 'guarda' nella griglia cosa c'è nelle coordinate in cui vuole andare: se trova il carattere spazio, procede e aggiorna la matrice spostando la sua @; se trova il carattere

del muro, l'azione di spostamento viene bloccata alla radice. I miei 0, 1 e -1 sono diventati direttamente i caratteri visivi spazio, % e #!"